

ANSI SQL using MS-SQL Server

Level-1 and 2 Problem: Create Database **EventDb** and create tables as per schema given below.

 Requirements

1. Add entity class **UserInfo** and add public properties:

Property Name	Type	Constraint
EmailId	varchar	PK
UserName	varchar	Not Null, Length: 1 to 50 char.
Role	varchar	Not Null, (Admin,Participant)-CHECK
Password	varchar	Not Null Length: 6 to 20 char

2. Add entity class **EventDetails** and add below public properties.

Property Name	Type	Constraint
EventId	int	PK
EventName	varchar	Not Null, Length: 1 to 50 char.
EventCategory	varchar	Not Null, Length: 1 to 50 char.
EventDate	datetime	Not Null
Description	varchar	Nullable
Status	varchar	Active or In-Active, CHECK

4. Add entity class **SpeakersDetails** and add below public properties.

Property Name	Type	Constraints
SpeakerId	int	PK
SpeakerName	varchar	Not Null, Length: 1 to 50 char.

5. Add entity class **SessionInfo** and add below public properties.

Property Name	Type	Constraints
SessionId	int	PK
EventId	int	Not Null (FK)

SessionTitle	varchar	Not Null, Length: 1 to 50 char.
SpeakerId	int	Not Null (FK)
Description	varchar	Nullable
SessionStart	datetime	Not Null (Time Slot)
SessionEnd	datetime	Not Null (Time Slot)
SessionUrl	varchar	

6. Add entity class **ParticipantEventDetails** and add below public properties.

Property Name	Type	Constraints
Id	int	(PK)
ParticipantEmailId	varchar	Not Null (FK)
EventId	int	Not Null (FK)
SessionId	Int	Not Null (FK)
IsAttended	bit	Yes or No, CHECK

Technical Constraints

- Use SQL Server.
- Use appropriate data types (INT, VARCHAR, DATE, DECIMAL).
- Follow normalization up to 2NF.
- Use ANSI SQL syntax only.

Expectations:

- Proper table structure with relationships.
- Valid constraints implementation.
- Correct data insertion and retrieval queries.

Learning Outcome

You will understand database structure, basic normalization, constraints, and ANSI SQL queries.

Code:

```
CREATE DATABASE EventDb;
```

```
GO
```

```
USE EventDb;
```

```
GO
```

```
CREATE TABLE UserInfo (
```

```
    EmailId VARCHAR(100) PRIMARY KEY,
```

```
    UserName VARCHAR(50) NOT NULL,
```

```
    Role VARCHAR(20) NOT NULL,
```

```
    Password VARCHAR(20) NOT NULL,
```

```
    CONSTRAINT CK_UserName_Length
```

```
        CHECK (LEN(UserName) BETWEEN 1 AND 50),
```

```
    CONSTRAINT CK_User_Role
```

```
        CHECK (Role IN ('Admin', 'Participant')),
```

```
    CONSTRAINT CK_Password_Length
```

```
        CHECK (LEN>Password) BETWEEN 6 AND 20)
```

```
);
```

```
CREATE TABLE EventDetails (
```

```
    EventId INT PRIMARY KEY,
```

```
    EventName VARCHAR(50) NOT NULL,
```

```
    EventCategory VARCHAR(50) NOT NULL,
```

```
    EventDate DATETIME NOT NULL,
```

Description VARCHAR(255),
Status VARCHAR(20) NOT NULL,

CONSTRAINT CK_EventName_Length
CHECK (LEN(EventName) BETWEEN 1 AND 50),

CONSTRAINT CK_EventCategory_Length
CHECK (LEN(EventCategory) BETWEEN 1 AND 50),

CONSTRAINT CK_Event_Status
CHECK (Status IN ('Active', 'In-Active'))

);

CREATE TABLE SpeakersDetails (
SpeakerId INT PRIMARY KEY,
SpeakerName VARCHAR(50) NOT NULL,

CONSTRAINT CK_SpeakerName_Length
CHECK (LEN(SpeakerName) BETWEEN 1 AND 50)

);

CREATE TABLE SessionInfo (
SessionId INT PRIMARY KEY,
EventId INT NOT NULL,
SessionTitle VARCHAR(50) NOT NULL,
SpeakerId INT NOT NULL,
Description VARCHAR(255),
SessionStart DATETIME NOT NULL,
SessionEnd DATETIME NOT NULL,
SessionUrl VARCHAR(255),

CONSTRAINT FK_Session_Event

FOREIGN KEY (EventId) REFERENCES EventDetails(EventId),

CONSTRAINT FK_Session_Speaker

FOREIGN KEY (SpeakerId) REFERENCES SpeakersDetails(SpeakerId),

CONSTRAINT CK_SessionTitle_Length

CHECK (LEN(SessionTitle) BETWEEN 1 AND 50),

CONSTRAINT CK_Session_Time

CHECK (SessionEnd > SessionStart)

);

CREATE TABLE ParticipantEventDetails (

Id INT PRIMARY KEY,

ParticipantEmailId VARCHAR(100) NOT NULL,

EventId INT NOT NULL,

SessionId INT NOT NULL,

IsAttended BIT NOT NULL,

CONSTRAINT FK_Participant_User

FOREIGN KEY (ParticipantEmailId) REFERENCES UserInfo(EmailId),

CONSTRAINT FK_Participant_Event

FOREIGN KEY (EventId) REFERENCES EventDetails(EventId),

CONSTRAINT FK_Participant_Session

FOREIGN KEY (SessionId) REFERENCES SessionInfo(SessionId)

);

INSERT INTO UserInfo VALUES

```
('admin@gmail.com','AdminUser','Admin','admin123'),  
('guru@gmail.com','Gurunath','Participant','guru123');
```

```
INSERT INTO EventDetails VALUES
```

```
(1,'Tech Conference','Technology','2026-05-10',  
 'AI and Cloud Event','Active');
```

```
INSERT INTO SpeakersDetails VALUES
```

```
(101,'Dr. Ravi Kumar');
```

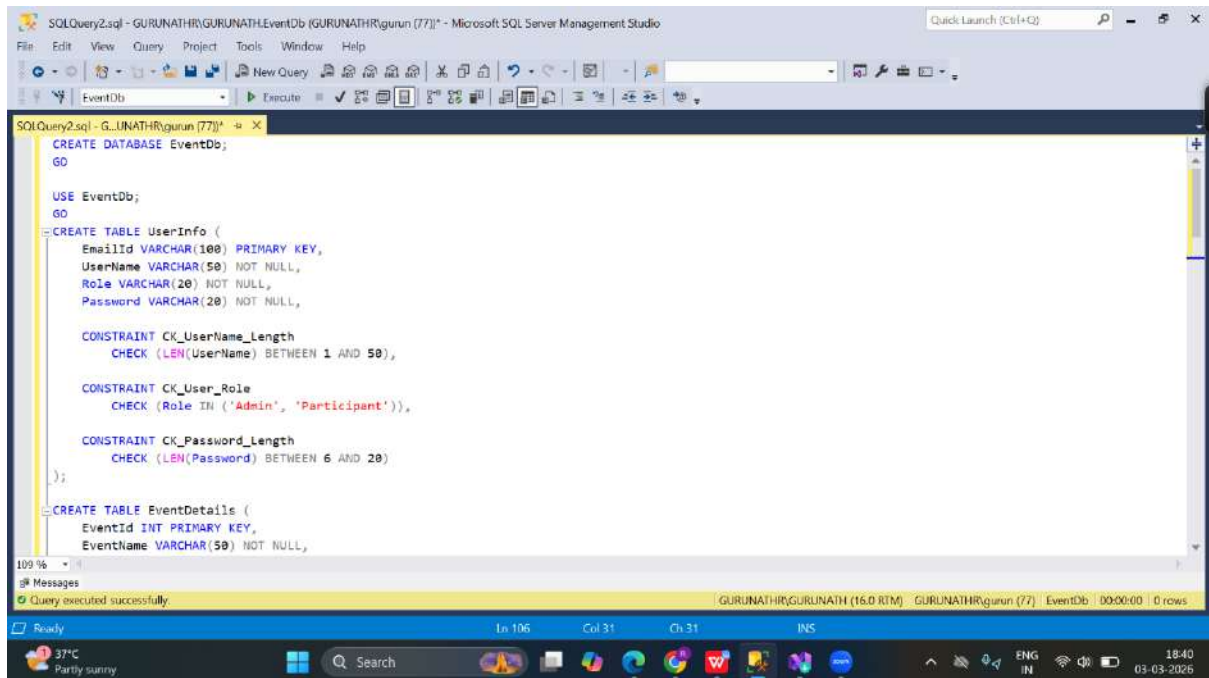
```
INSERT INTO SessionInfo VALUES
```

```
(1001,1,'AI Trends',101,  
 'AI Session','2026-05-10 10:00:00',  
 '2026-05-10 11:00:00',  
 'https://sessionlink.com');
```

```
INSERT INTO ParticipantEventDetails VALUES
```

```
(1,'guru@gmail.com',1,1001,1);
```

Code Screenshots:



```
SQLQuery2.sql - GURUNATHR\GURUNATH\EventDb (GURUNATHR\gurun (77)) - Microsoft SQL Server Management Studio
File Edit View Query Project Tools Window Help
New Query
EventDb Execute
SQLQuery2.sql - GURUNATHR\gurun (77)
CREATE DATABASE EventDb;
GO

USE EventDb;
GO

CREATE TABLE UserInfo (
    EmailId VARCHAR(100) PRIMARY KEY,
    UserName VARCHAR(50) NOT NULL,
    Role VARCHAR(20) NOT NULL,
    Password VARCHAR(20) NOT NULL,

    CONSTRAINT CK_UserName_Length
        CHECK (LEN(UserName) BETWEEN 1 AND 50),

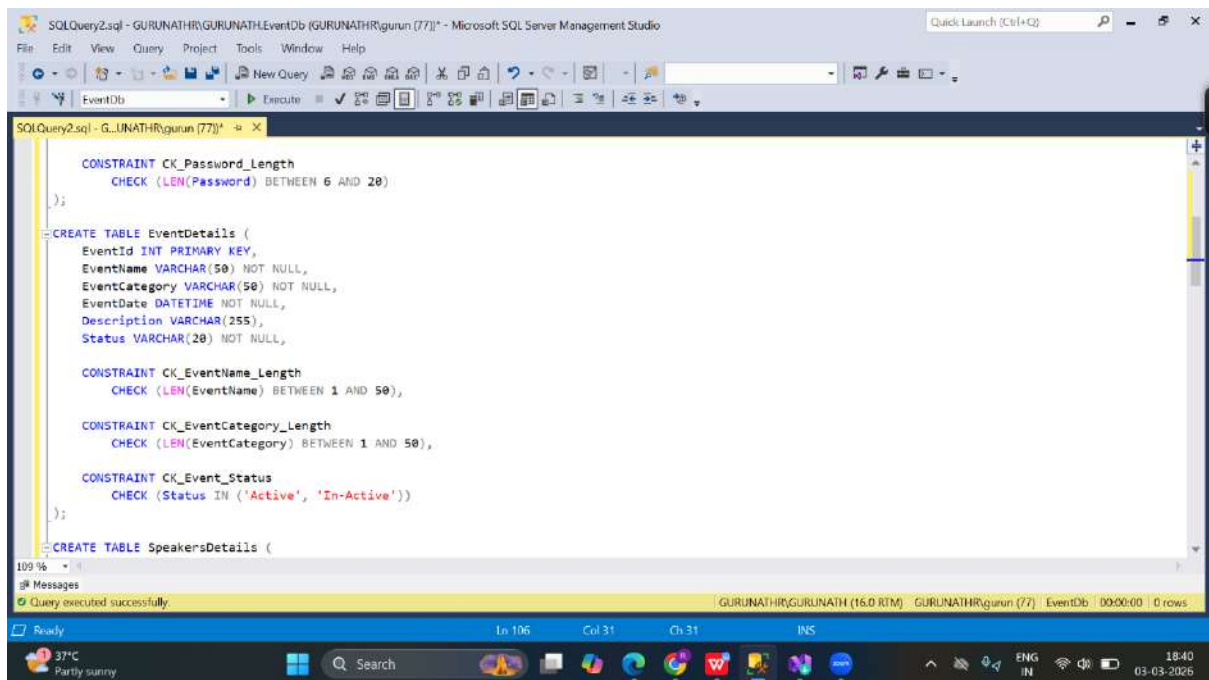
    CONSTRAINT CK_User_Role
        CHECK (Role IN ('Admin', 'Participant')),

    CONSTRAINT CK_Password_Length
        CHECK (LEN>Password) BETWEEN 6 AND 20)
);

CREATE TABLE EventDetails (
    EventId INT PRIMARY KEY,
    EventName VARCHAR(50) NOT NULL,
```

109 %
Messages
Query executed successfully.
GURUNATHR\GURUNATH (16.0 RTM) GURUNATHR\gurun (77) EventDb 00:00:00 0 rows

Ready In 106 Col 31 Ch 31 INS
37°C Partly sunny Search ENG IN 18:40 03-03-2026



```
SQLQuery2.sql - GURUNATHR\GURUNATH\EventDb (GURUNATHR\gurun (77)) - Microsoft SQL Server Management Studio
File Edit View Query Project Tools Window Help
New Query
EventDb Execute
SQLQuery2.sql - GURUNATHR\gurun (77)
    CONSTRAINT CK_Password_Length
        CHECK (LEN>Password) BETWEEN 6 AND 20)
);

CREATE TABLE EventDetails (
    EventId INT PRIMARY KEY,
    EventName VARCHAR(50) NOT NULL,
    EventCategory VARCHAR(50) NOT NULL,
    EventDate DATETIME NOT NULL,
    Description VARCHAR(255),
    Status VARCHAR(20) NOT NULL,

    CONSTRAINT CK_EventName_Length
        CHECK (LEN(EventName) BETWEEN 1 AND 50),

    CONSTRAINT CK_EventCategory_Length
        CHECK (LEN(EventCategory) BETWEEN 1 AND 50),

    CONSTRAINT CK_Event_Status
        CHECK (Status IN ('Active', 'In-Active'))
);

CREATE TABLE SpeakersDetails (
```

109 %
Messages
Query executed successfully.
GURUNATHR\GURUNATH (16.0 RTM) GURUNATHR\gurun (77) EventDb 00:00:00 0 rows

Ready In 106 Col 31 Ch 31 INS
37°C Partly sunny Search ENG IN 18:40 03-03-2026

SQLQuery2.sql - GURUNATH\GURUNATH.EventDb (GURUNATH\gurun (77)) - Microsoft SQL Server Management Studio

File Edit View Query Project Tools Window Help

Quick Launch (Ctrl+Q)

EventDb

SQLQuery2.sql - GURUNATH\gurun (77)* X

```
CONSTRAINT CK_EventCategory_Length
CHECK (LEN(EventCategory) BETWEEN 1 AND 50),

CONSTRAINT CK_Event_Status
CHECK (Status IN ('Active', 'In-Active'))
);

CREATE TABLE SpeakersDetails (
SpeakerId INT PRIMARY KEY,
SpeakerName VARCHAR(50) NOT NULL,

CONSTRAINT CK_SpeakerName_Length
CHECK (LEN(SpeakerName) BETWEEN 1 AND 50)
);

CREATE TABLE SessionInfo (
SessionId INT PRIMARY KEY,
EventId INT NOT NULL,
SessionTitle VARCHAR(50) NOT NULL,
SpeakerId INT NOT NULL,
Description VARCHAR(255),
SessionStart DATETIME NOT NULL,
SessionEnd DATETIME NOT NULL,
SessionUrl VARCHAR(255),
```

109 %

Messages

Query executed successfully.

GURUNATH\GURUNATH (16.0 RTM) GURUNATH\gurun (77) EventDb 00:00:00 0 rows

Ready In 106 Col 31 Ch 31 INS

37°C Partly sunny

Search

ENG IN 18:40 03-03-2026

SQLQuery2.sql - GURUNATH\GURUNATH.EventDb (GURUNATH\gurun (77)) - Microsoft SQL Server Management Studio

File Edit View Query Project Tools Window Help

Quick Launch (Ctrl+Q)

EventDb

SQLQuery2.sql - GURUNATH\gurun (77)* X

```
SessionId INT PRIMARY KEY,
EventId INT NOT NULL,
SessionTitle VARCHAR(50) NOT NULL,
SpeakerId INT NOT NULL,
Description VARCHAR(255),
SessionStart DATETIME NOT NULL,
SessionEnd DATETIME NOT NULL,
SessionUrl VARCHAR(255),

CONSTRAINT FK_Session_Event
FOREIGN KEY (EventId) REFERENCES EventDetails(EventId),

CONSTRAINT FK_Session_Speaker
FOREIGN KEY (SpeakerId) REFERENCES SpeakersDetails(SpeakerId),

CONSTRAINT CK_SessionTitle_Length
CHECK (LEN(SessionTitle) BETWEEN 1 AND 50),

CONSTRAINT CK_Session_Time
CHECK (SessionEnd > SessionStart)
);

CREATE TABLE ParticipantEventDetails (
Id INT PRIMARY KEY,
```

109 %

Messages

Query executed successfully.

GURUNATH\GURUNATH (16.0 RTM) GURUNATH\gurun (77) EventDb 00:00:00 0 rows

Ready In 105 Col 31 Ch 31 INS

Search

ENG IN 18:41 03-03-2026

SQLQuery2.sql - GURUNATH\GURUNATH.EventDb (GURUNATH\gurun (77)) - Microsoft SQL Server Management Studio

File Edit View Query Project Tools Window Help

Quick Launch (Ctrl+Q)

EventDb

SQLQuery2.sql - G...UNATH\gurun (77)* X

```
        CHECK (LEN(SessionTitle) BETWEEN 1 AND 50),

        CONSTRAINT CK_Session_Time
        CHECK (SessionEnd > SessionStart)
    );

CREATE TABLE ParticipantEventDetails (
    Id INT PRIMARY KEY,
    ParticipantEmailId VARCHAR(100) NOT NULL,
    EventId INT NOT NULL,
    SessionId INT NOT NULL,
    IsAttended BIT NOT NULL,

    CONSTRAINT FK_Participant_User
        FOREIGN KEY (ParticipantEmailId) REFERENCES UserInfo(EmailId),

    CONSTRAINT FK_Participant_Event
        FOREIGN KEY (EventId) REFERENCES EventDetails(EventId),

    CONSTRAINT FK_Participant_Session
        FOREIGN KEY (SessionId) REFERENCES SessionInfo(SessionId)
);

INSERT INTO UserInfo VALUES
```

109 %

Messages

Query executed successfully.

GURUNATH\GURUNATH (16.0 RTM) GURUNATH\gurun (77) EventDb 00:00:00 0 rows

Ready In 105 Col 31 Ch 31 INS

Search

ENG IN 18:41 03-03-2026

SQLQuery2.sql - GURUNATH\GURUNATH.EventDb (GURUNATH\gurun (77)) - Microsoft SQL Server Management Studio

File Edit View Query Project Tools Window Help

Quick Launch (Ctrl+Q)

EventDb

SQLQuery2.sql - G...UNATH\gurun (77)* X

```
        CONSTRAINT FK_Participant_Event
        FOREIGN KEY (EventId) REFERENCES EventDetails(EventId),

        CONSTRAINT FK_Participant_Session
        FOREIGN KEY (SessionId) REFERENCES SessionInfo(SessionId)
    );

INSERT INTO UserInfo VALUES
    ('admin@gmail.com', 'AdminUser', 'Admin', 'admin123'),
    ('guru@gmail.com', 'Gurunath', 'Participant', 'guru123');

INSERT INTO EventDetails VALUES
    (1, 'Tech Conference', 'Technology', '2026-05-10',
     'AI and Cloud Event', 'Active');

INSERT INTO SpeakersDetails VALUES
    (101, 'Dr. Ravi Kumar');

INSERT INTO SessionInfo VALUES
    (1001, 1, 'AI Trends', 101,
     'AI Session', '2026-05-10 10:00:00',
     '2026-05-10 11:00:00',
     'https://sessionlink.com');
```

109 %

Messages

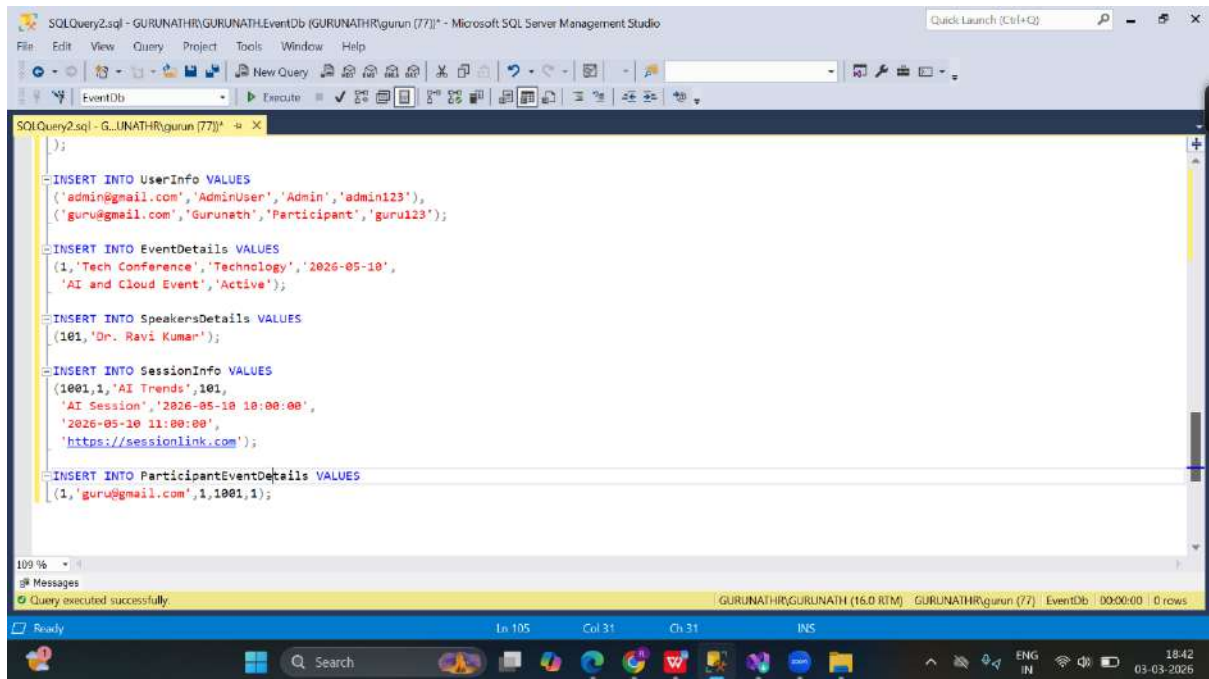
Query executed successfully.

GURUNATH\GURUNATH (16.0 RTM) GURUNATH\gurun (77) EventDb 00:00:00 0 rows

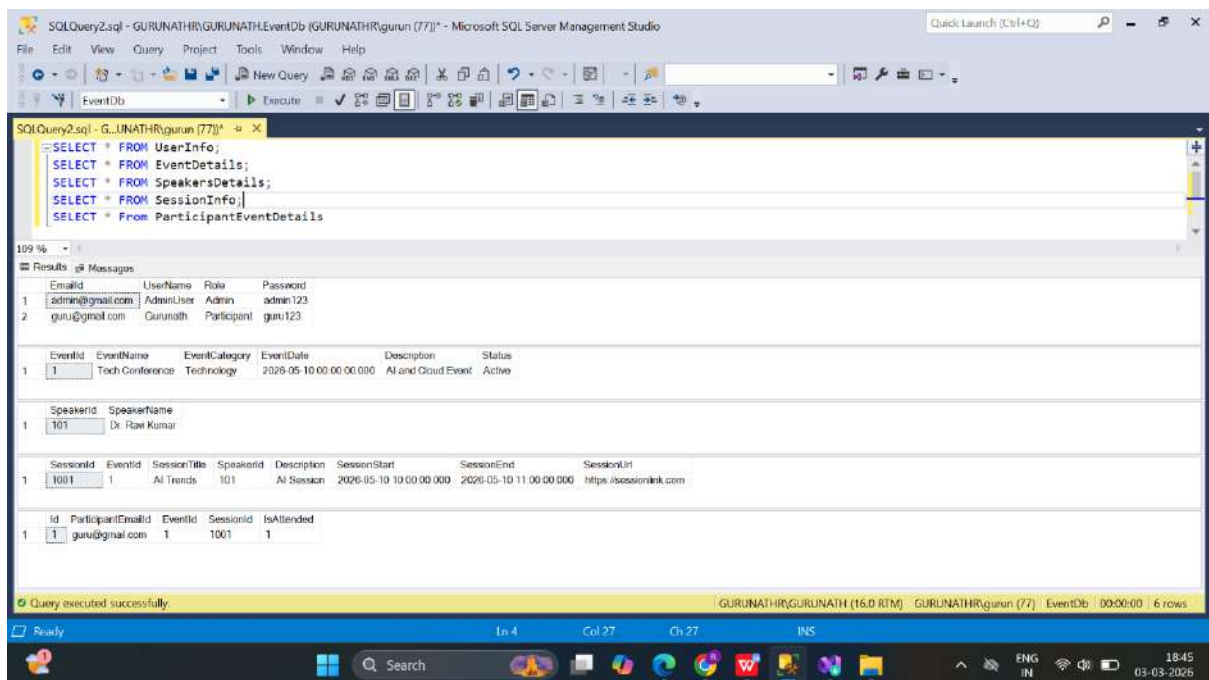
Ready In 105 Col 31 Ch 31 INS

Search

ENG IN 18:41 03-03-2026



Output Screenshots:



Explanation: In **Microsoft SQL Server**, the tables created inside EventDb are structured using Primary Keys, Foreign Keys, and CHECK constraints to enforce data integrity and relationships.

The schema follows normalization principles (up to 2NF), ensuring minimal redundancy and proper relational mapping between events, users, sessions, and participants.